

10 Proven Websites Prompts

Copy any prompt, fill the [BRACKETS], send. Mined from 500+ hours of top Claude builders.

1

Three Layout Variants First

Front-end is a visual medium; you must SEE macro layout options to choose well — this mimics the compare-variants workflow of pro design tools.

I want to build a website for [BUSINESS / PURPOSE]. Don't give me just one layout — build me 3 genuinely different variants (for example editorial, bold/dark, and minimalist). Let me click through all three and see them side by side before we pick a direction and dive into components. Use the front-end design skill.

2

Always Invoke the Front-End Design Skill

The same prompt with the design skill produces a polished, branded site vs a generic one — it's the difference between vibe-coded and professional.

Build me a modern, professional website for [BUSINESS]. Use the front-end design skill — I do not want generic AI-slop design. Pick a distinctive Google Font (never Inter), give cards real visual weight, use premium spacing, and never use a plain white background with a purple/blue gradient.

3

Clone a Reference Site's Code

Screenshots alone get you ~50% there; giving Claude the actual source code closes the gap and lets you near-pixel-perfect replicate a \$10k design.

I want to clone the design of a website I admire. I'm pasting in its full HTML, the CSS/JS it links to, full-page screenshots, and the URL: [PASTE HTML + SCREENSHOTS + URL]. Recreate this site's layout and feel as a template — then we'll swap in my own branding, copy, colors and logo so it becomes mine.

4

Brand Assets Build

Tagging real brand assets directly makes the site feel genuinely branded instead of generic, from a single prompt.

Build me a modern, professional landing page for [BUSINESS]. Here is my logo @logo.png and my brand guidelines @brand-guidelines.png — make the whole page branded with that logo, those exact colors, and that typography throughout. Use the front-end design skill and spin up a dev server when finished so I can review it.

5

Section-by-Section Build

Section-by-section building avoids overwhelm and keeps Claude from losing track of multiple simultaneous changes.

Let's build this website one section at a time, not all at once. Start with just the hero section. When it's done and I've approved it, we'll move to section one, then section two. For each section, suggest what dynamic element or visual would fit best, then build it. Make only one set of changes per prompt.

6

Screenshot Self-Correction Loop

Claude can see — visual self-correction produces a far better first version than a blind one-shot; disabling it for animations prevents over-engineering loops.

After you build this page, run a screenshot loop: take a screenshot of what you built, compare it against the design goal, fix any mismatches, and repeat — do about three build-and-screenshot passes before you even show me V1. Note: if a section has an animated background, do NOT screenshot it — just work in the code and I'll review that part.

7

Browser QA the Whole Site

Automates the laborious manual QA of a site into a self-fixing loop — Claude finds and patches bugs (stale overlays, broken enter-key, missed states) before a customer hits them.

Spin up a dev server so you can actually run this site, then use a headed browser so I can watch — click through every page, fill in every form field, and test every interaction. Note any bugs or functionality problems, fix the actual site, then re-test. Loop until everything passes.

8

Pull a Polished Component

Small premium details elevate an entire page; importing a real, vetted component beats hoping Claude invents one.

I want to upgrade one piece of this page with a more premium component. Here is the code/prompt for a component I copied from a component library (21st.dev / shadcn / similar): [PASTE COMPONENT CODE]. Work it in precisely behind the hero text (or wherever I specify) and follow the code exactly — small premium flourishes like this make the whole page feel like I actually cared.

9

Mobile Optimization Pass

Claude Code does not auto-optimize for mobile; this commonly-missed step is what breaks deployed sites on phones.

This site is built and looks good on desktop, but it is NOT optimized for mobile. Open it, switch to a phone-sized viewport, and restructure the layout so it works beautifully on phones — readable text, tappable buttons, no horizontal scroll — without breaking the desktop version. Show me before and after.

10

Ship It (GitHub + Vercel)

The full free ship pipeline — verify locally, then GitHub + Vercel for one-click deploy and auto-updating production.

First confirm the site works by opening it on localhost for me to review. Then create a new private GitHub repo for it and push the codebase. After that, walk me through connecting the repo to Vercel so it deploys live. Set a rule for this project: we always test on localhost until I explicitly say 'push to GitHub.'